

AgentOpsHub

Developer Guide

Multi-Agent Enterprise Copilot Framework

.NET 9 | Azure AI Foundry | CopilotKit | AG-UI Protocol

v1.0 | June 2026

Table of Contents

1. Introduction
2. Architecture Overview
3. Project Structure
4. How It Works End-to-End
5. Core Abstractions
6. The Agent Tool Loop
7. MCP: Data Connections
8. Building Your First Agent
9. Building an MCP Client
10. Framework Features
11. Configuration Reference
12. Quick Start

1. Introduction

AgentOpsHub is a **multi-agent enterprise copilot framework** built with .NET 9, Azure AI Foundry, and CopilotKit. It provides the scaffolding to build, deploy, and operate AI agents that can reason over data, call tools autonomously, and stream responses to a chat UI via the AG-UI protocol.

The framework is designed around three principles:

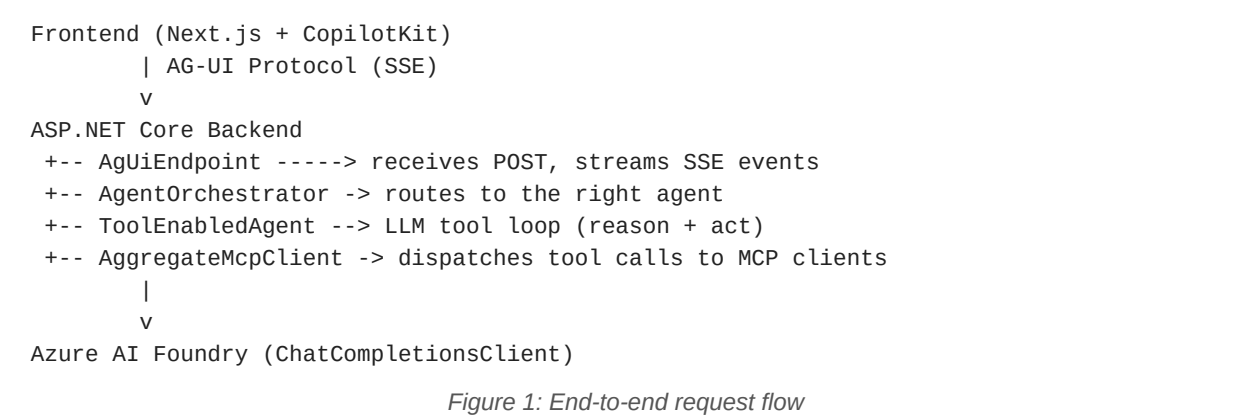
- **Separation of concerns** — The core engine, data connections, and agent logic each live in their own folder and never cross boundaries.
- **Agent autonomy** — Agents decide which tools to call and when, using an LLM-driven tool loop rather than hardcoded sequences.
- **Extensibility** — Adding a new agent or data source is a single class plus one line in Program.cs. No orchestrator changes needed.

What You Get Out of the Box

- AG-UI SSE endpoint for CopilotKit integration
- LLM-based routing with keyword fast-path
- Agentic tool loop (LLM decides tool calls iteratively)
- Aggregate MCP client supporting multiple data sources
- Conversation memory (full history passed to agents)
- Structured logging and observability
- HTTP resilience with automatic retries

2. Architecture Overview

The system has four layers. The frontend sends user messages over SSE. The backend routes them to the right agent, which uses an LLM to reason and call tools. Tools are provided by MCP clients that connect to external APIs.



Key Components

Component	Location	Responsibility
AgUiEndpoint	Core/Endpoints/	HTTP handler; SSE streaming; conversation memory
AgentOrchestrator	Core/Services/	Routes messages to agents (keyword + LLM fallback)
ToolEnabledAgentBase	Core/Agents/	Abstract base class implementing the agentic tool loop
AggregateMcpClient	Core/Services/	Composites multiple IMcpClient instances by skill name
IAgentService	Abstractions/	Interface every agent implements
IMcpClient	Abstractions/	Interface every data connection implements

3. Project Structure

Three concerns, three folders. Each has a clear responsibility and dependency direction:

```
src/
+-- AgentOpsHub/                # Core engine (FRAMEWORK)
|   +-- AgentOpsHub.Abstractions/ # Interfaces, models (zero deps)
|   +-- AgentOpsHub.Core/        # Endpoint, orchestrator, tool loop
|
+-- MCP/                        # Data connections
|   +-- MCP.MarketData/         # SEC EDGAR + Alpha Vantage
|   +-- MCP.Weather/           # (add your own)
|
+-- MyAgents/                   # Custom agent logic
|   +-- MyFirstAgent/          # Earnings Surprise Scanner
|   +-- MySecondAgent/        # (add your own)
|
+-- AgentOpsHub.Web/            # Thin composition root

frontend/                       # Next.js + CopilotKit chat UI
```

Dependency Direction

Dependencies always flow **inward** toward Abstractions. Agents and MCP clients reference the framework; the framework never references them.

```
AgentOpsHub.Web ---> Core ---> Abstractions <--- MCP.MarketData
                  ^                <--- MyFirstAgent
                  |
                MyFirstAgent (inherits ToolEnabledAgentBase from Core)
```

Abstractions has zero NuGet dependencies — it is entirely SDK-agnostic. This means you can swap Azure AI Foundry for OpenAI, Ollama, or any other provider by changing Core only. Agent logic and MCP clients never touch LLM SDKs directly.

4. How It Works End-to-End

Here is what happens when a user types a message in the CopilotKit chat UI:

- 1. Frontend sends POST** — CopilotKit sends a POST request to `/api/copilotkit` with the full message history as a JSON array.
- 2. AgUiEndpoint receives it** — Extracts the last user message, builds conversation history into an `AgentRequest`, and sets up SSE streaming headers.
- 3. AgentOrchestrator routes** — Tries each agent's `canHandle()` method (fast keyword match). If none match, falls back to LLM intent classification using agent names and descriptions.
- 4. Agent.StreamAsync() runs** — The selected agent starts its tool loop. The base class builds the initial message list from history + system prompt + user message.
- 5. LLM decides tool calls** — The LLM sees the available tools (from MCP skills) and either returns `tool_calls` or a text response.
- 6. Tools are executed** — For each tool call, the framework calls `IMcpClient.ExecuteAsync()` and adds the result back to the message list.
- 7. Loop continues** — Steps 5–6 repeat until the LLM produces a final text answer (or hits `MaxToolIterations`).
- 8. Response streams** — The final text response is streamed via SSE events (`text.message.content`) back to the frontend.

5. Core Abstractions

The Abstractions project defines the contracts that all agents and data connections implement. It has **zero external dependencies**, making it the stable foundation of the framework.

IAgentService

Every agent implements this interface. It defines identity, routing, and execution:

```
public interface IAgentService
{
    string Name { get; }
    string Description { get; }
    bool CanHandle(string userMessageLower);
    Task<AgentResponse> HandleAsync(AgentRequest request, CancellationToken ct);
    IAsyncEnumerable<string> StreamAsync(AgentRequest request, CancellationToken ct);
}
```

IMcpClient

Every data connection implements this interface. It exposes tools that agents can call:

```
public interface IMcpClient
{
    Task<string> ExecuteAsync(string toolName, string paramsJson, CancellationToken ct);
    IReadOnlyList<SkillDefinition> ListSkills();
}

public sealed record SkillDefinition(
    string Name,
    string Description,
    IReadOnlyDictionary<string, string> Parameters);
```

AgentRequest & AgentResponse

SDK-agnostic request/response models that flow through the system:

```
public sealed class AgentRequest
{
    public required string ThreadId { get; init; }
    public required string RunId { get; init; }
    public required string UserMessage { get; init; }
    public List<ChatMessage> History { get; init; } = [];
}

public sealed class AgentResponse
{
    public required string Content { get; init; }
```

```
    public string? AgentName { get; init; }  
    public Dictionary<string, object>? Metadata { get; init; }  
}
```

ToolDefinition

SDK-agnostic representation of a tool for LLM function-calling. The `ParametersJsonSchema` field stores a raw JSON Schema string so Abstractions stays decoupled from any LLM SDK. Core converts this to the SDK-specific format.

```
public sealed class ToolDefinition  
{  
    public required string Name { get; init; }  
    public required string Description { get; init; }  
    public required string ParametersJsonSchema { get; init; }  
}
```

6. The Agent Tool Loop

The **ToolEnabledAgentBase** class in Core is the heart of the framework. It implements the agentic loop that makes agents truly autonomous — the LLM decides which tools to call and when, rather than following a hardcoded sequence.

How the Loop Works

1. Build messages: [SystemPrompt] + [History] + [UserMessage]
2. Add tools: Convert MCP skills -> ToolDefinitions -> SDK tool format
3. Call LLM: Send messages + tools to ChatCompletionsClient
4. Check response:
 - If tool_calls: Execute each via IMcpClient, add results, GOTO 3
 - If text: Stream the final answer to the user, DONE
5. Safety: MaxToolIterations (default 10) prevents runaway loops

What Subclasses Override

Method	Required	Purpose
Name	Yes	Agent identity string
Description	Yes	Used by LLM routing to match intent
CanHandle()	Yes	Fast keyword matching for routing
GetSystemPrompt()	Yes	Returns the system prompt for this agent's task
GetAvailableTools()	No	Filter which MCP tools this agent can use (default: all)
MaxToolIterations	No	Override the safety limit (default: 10)

Why This Matters

Before this pattern, agents had to manually orchestrate tool calls: "call SEC EDGAR, then call stock quote, then call RSI, then format results, then call LLM." That is a **prompt wrapper**, not an agent.

With the tool loop, you just tell the LLM what data to gather and how to analyze it. The LLM figures out the sequence, handles errors, asks follow-up questions, and produces a final answer. This is what makes it a true **agent**.

7. MCP: Data Connections

MCP (Model Context Protocol) clients are the bridge between agents and external data. Each client exposes a set of named **skills** that agents can call through the tool loop. Agents never make HTTP calls directly — they go through IMcpClient.

AggregateMcpClient

The framework supports **multiple MCP clients** registered simultaneously. The AggregateMcpClient in Core composites them into a single IMcpClient that agents receive. It dispatches tool calls by skill name — the first client that registered a given skill handles it.

```
// Register multiple MCP clients in Program.cs
builder.Services.AddMarketDataMcpClient(); // SEC EDGAR + Alpha Vantage
builder.Services.AddWeatherMcpClient();    // OpenWeatherMap
builder.Services.AddCalendarMcpClient();   // Google Calendar

// All skills from all clients are available to all agents
```

Built-in: MCP.MarketData

The included MarketDataMcpClient provides three skills for financial data:

Skill Name	Source	Description
edgar_company_facts	SEC EDGAR	Fetch financial filings (EPS, revenue, assets) by ticker
stock_quote	Alpha Vantage	Get current stock price and daily change
stock_technicals	Alpha Vantage	Get RSI, SMA, EMA, MACD indicators

Resilience

MCP.MarketData uses **Microsoft.Extensions.Http.Resilience** for automatic retries with exponential backoff. Named HttpClient ("SEC", "AlphaVantage") are configured with timeouts and the standard resilience handler. Failed calls return structured error JSON instead of throwing — the LLM can see the error and decide how to proceed.

8. Building Your First Agent

Building a new agent takes three steps: create a class library, implement the agent, and register it in Program.cs. The entire process takes about 5 minutes.

Step 1: Create the Project

```
# Create a new class library under src/MyAgents/  
dotnet new classlib -n MySecondAgent -o src/MyAgents/MySecondAgent  
  
# Add a reference to Core (gives you Abstractions + ToolEnabledAgentBase)  
cd src/MyAgents/MySecondAgent  
dotnet add reference ../../AgentOpsHub/AgentOpsHub.Core/AgentOpsHub.Core.csproj  
  
# Add project to the solution  
cd ../../..  
dotnet sln add src/MyAgents/MySecondAgent/MySecondAgent.csproj
```

Step 2: Implement the Agent

```
using AgentOpsHub.Abstractions;  
using AgentOpsHub.Core.Agents;  
using Azure.AI.Inference;  
using Microsoft.Extensions.Logging;  
  
namespace MySecondAgent;  
  
public sealed class PortfolioAnalysisAgent : ToolEnabledAgentBase  
{  
    public override string Name => "PortfolioAnalysis";  
    public override string Description =>  
        "Analyzes a stock portfolio for risk, diversification, and performance."  
  
    public PortfolioAnalysisAgent(  
        ChatCompletionsClient chatClient,  
        IMcpClient mcpClient,  
        ILogger<PortfolioAnalysisAgent> logger)  
        : base(chatClient, mcpClient, logger) { }  
  
    public override bool CanHandle(string userMessageLower) =>  
        userMessageLower.Contains("portfolio") ||  
        userMessageLower.Contains("diversif") ||  
        userMessageLower.Contains("risk analysis");  
  
    protected override string GetSystemPrompt(AgentRequest request) => ""  
        You are a portfolio analyst. Use the available tools to:  
        1. Look up fundamentals for each ticker in the user's portfolio  
        2. Get current prices and technical indicators
```

```
        3. Assess sector concentration, valuation risk, and momentum
        4. Produce a diversification score and actionable recommendations
        Output a markdown report with tables and clear reasoning.
        """;
    }
```

Step 3: Register in Program.cs

```
// Add one line to Program.cs:
builder.Services.AddSingleton<IAgentService, PortfolioAnalysisAgent>();
```

That is it. The agent is automatically:

- Discovered by the AgentOrchestrator for routing
- Given all MCP tools from all registered clients
- Equipped with conversation memory
- Streaming responses via SSE to the frontend

9. Building an MCP Client

MCP clients connect your agents to external data. Each client is a class library that implements `IMcpClient` and defines a set of skills.

Step 1: Create the Project

```
dotnet new classlib -n MCP.Weather -o src/MCP/MCP.Weather
cd src/MCP/MCP.Weather
dotnet add reference ../../AgentOpsHub/AgentOpsHub.Abstractions
dotnet add package Microsoft.Extensions.Http.Resilience
```

Step 2: Implement `IMcpClient`

```
public sealed class WeatherMcpClient : IMcpClient
{
    private static readonly IReadOnlyList<SkillDefinition> Skills =
    [
        new("get_weather", "Get current weather for a city",
            new Dictionary<string, string>
            {
                ["city"] = "City name (e.g. London, New York)"
            },
        );

    public IReadOnlyList<SkillDefinition> ListSkills() => Skills;

    public async Task<string> ExecuteAsync(
        string toolName, string paramsJson, CancellationToken ct)
    {
        return toolName switch
        {
            "get_weather" => await FetchWeather(paramsJson, ct),
            _ => JsonSerializer.Serialize(new { error = "Unknown" }),
        };
    }
}
```

Step 3: Create a Registration Extension

```
public static class WeatherServiceExtensions
{
    public static IServiceCollection AddWeatherMcpClient(
        this IServiceCollection services)
    {
    }
```

```
        services.AddHttpClient("Weather", client =>
            client.Timeout = TimeSpan.FromSeconds(10))
            .AddStandardResilienceHandler();

        services.AddKeyedSingleton<IMcpClient,
            WeatherMcpClient>("mcp-individual");
        return services;
    }
}
```

Step 4: Register in Program.cs

```
builder.Services.AddWeatherMcpClient();
// Skills automatically appear in AggregateMcpClient
```

10. Framework Features

Conversation Memory

The AG-UI endpoint automatically populates `AgentRequest.History` from the full message array sent by CopilotKit. Agents receive the complete conversation context, enabling multi-turn interactions. The `ToolEnabledAgentBase` includes this history in the LLM message list so the model has full context.

LLM-Based Routing

The `AgentOrchestrator` uses a two-tier routing strategy:

- **Fast path:** Each agent's `CanHandle()` method does keyword matching. If any agent matches, it is selected instantly with zero LLM cost.
- **Slow path:** If no agent matches, the orchestrator sends the user message to the LLM with a list of all agent names and descriptions. The LLM picks the best agent. This handles natural language queries that do not contain exact keywords.

Structured Logging

All framework components use `ILogger` with structured log properties. Key events logged:

- AG-UI request received (`ThreadId`, message count)
- Agent routing decisions (which agent, method used)
- Tool execution timing (tool name, elapsed ms)
- Streaming completion timing
- Errors and fallbacks with context

Error Resilience

MCP clients use `Microsoft.Extensions.Http.Resilience` for automatic retry with exponential backoff on transient HTTP failures. Failed tool calls return structured error JSON rather than throwing exceptions, allowing the LLM to handle failures gracefully (e.g., skip a ticker and continue with the rest).

11. Configuration Reference

All configuration lives in `appsettings.json` (or environment variables for Docker):

```
{
  "AIFoundry": {
    "Endpoint": "https://your-resource.openai.azure.com",
    "Key": "your-api-key",
    "DeploymentName": "gpt-4o",
    "UseAzureCredential": false
  },
  "AlphaVantage": {
    "ApiKey": "your-alpha-vantage-key"
  }
}
```

Setting	Required	Description
AIFoundry:Endpoint	Yes	Azure OpenAI or compatible endpoint URL
AIFoundry:Key	Conditional	API key (not needed if UseAzureCredential=true)
AIFoundry:DeploymentName	Yes	Model deployment name (e.g. gpt-4o)
AIFoundry:UseAzureCredential	No	Use DefaultAzureCredential (Managed Identity)
AlphaVantage:ApiKey	No	Alpha Vantage API key (default: 'demo')

Docker Compose

```
# Pass config via environment variables:
AIFOUNDRY_KEY=your-key docker compose up --build

# Or use appsettings.Development.json for local development
```

12. Quick Start

Prerequisites

- .NET 9 SDK
- Node.js 22+
- An Azure AI Foundry deployment (or OpenAI-compatible endpoint)
- Alpha Vantage API key (optional — 'demo' key works for limited tickers)

1. Clone and Configure

```
git clone <your-repo-url>
cd AgentOpsHub

# Edit src/AgentOpsHub.Web/appsettings.Development.json
# with your AIFoundry endpoint and key
```

2. Run the Backend

```
cd src/AgentOpsHub.Web
dotnet run
```

3. Run the Frontend

```
cd frontend
npm install
npm run dev
```

4. Try It

Open <http://localhost:3000> and try these prompts:

- "Scan for earnings surprises" — triggers the Earnings Surprise Scanner
- "Scan \$NVDA \$TSLA \$AAPL" — scans specific tickers
- "What can you help me with?" — tests LLM routing fallback

Composition Root

The final Program.cs is 12 lines — thin, declarative, no business logic:

```
using AgentOpsHub.Abstractions;
using AgentOpsHub.Core;
using MCP.MarketData;
using MyFirstAgent;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAgentOpsHub(builder.Configuration);
builder.Services.AddMarketDataMcpClient();
builder.Services.AddSingleton<IAgentService, EarningsSurpriseScannerAgent>();

var app = builder.Build();
app.UseAgentOpsHub();
app.Run();
```

AgentOpsHub — Built with .NET 9, Azure AI Foundry, CopilotKit, and the AG-UI Protocol.